

A Cognitive Loop for LLM Agents: Pre-Write Declaration as an Instrumentation Point for Transparency, Assistance, and Knowledge Reuse

Cognitive Workshop

August 2026 (v3: full rewrite)

Abstract

This is a system description and experience report. We are engineers, not LLM researchers. We built a cognitive management system for a coding agent (Claude Code) with one core goal: make the agent's working process transparent, auditable, and reusable. The central mechanism is a single discipline: before every file write, the agent must declare what it is doing—one sentence stating the purpose of the current step. This declaration turns the agent's otherwise implicit cognitive state into a real-time, structured, machine-readable data stream. Three consumers operate on that stream: a monitor (task drift, fix-loop detection, a three-tier escalation ending in external-model consultation), an assistant (knowledge and memory injection under strict anti-fatigue rules), and a consolidator (dual-track records with trust grading and causality tags, producing training corpus for future models—a data flywheel). The whole loop was ported to a second agent runtime (DeepSeek Harness) by sharing state files rather than copying code. We report production evidence (5 unsupervised consultations in real research sessions; zero-hint declaration compliance) and production failures (a deny gate that silently never worked; a reminder that nagged every turn; 76 consecutive false drift alarms). We have no quantitative benchmarks and make no performance claims. The system runs daily in a research-engineering environment; code and this paper are open.

1 Introduction

The problem. A coding agent's working process is opaque and linear. What it is currently trying to do, whether it has drifted from the original goal, where it is stuck, and what it learned this session are not visible from outside the conversation. When the session closes, the process is lost. Existing open-source effort concentrates on the memory layer—what to store and how to retrieve it (Mem0, Zep/Graphiti, Letta and the 2026 wave of memory systems). The memory layer answers “what does the agent know”. It does not answer “what is the agent doing, is it doing it well, and is the process being preserved”.

The approach. We organize the agent's work as a **cognitive loop**—six steps spanning a session's full lifecycle, with experience persisting on the loop between sessions. The pivot of the loop is a single mechanism we consider the most consequential design decision in the system:

Pre-write declaration. Before every file write, the agent must declare what the current step is doing. The declaration is written to a shared state file with a 300-second TTL. Missing, expired, or system-fabricated declarations are all rejected.

The value of this discipline is not that it restrains the agent. It is that the declaration gives the system *continuous access to the agent's latest cognitive state* as structured data. Without it, the agent's intent exists only as conversational text—noisy, delayed, and hard to use programmatically. With it, monitoring, assistance, and knowledge consolidation all have a real-time signal to consume.

What this paper is. A system description and experience report: the declaration protocol (Section 2), the loop it anchors (Section 3), the three consumers of the signal (Section 4), a cross-runtime port (Section 5), and an honest account of production failures (Section 6). We claim no quantitative superiority over any baseline; we report what ran, what broke, and what the mechanisms produced.

Safety gates (dangerous-command denial, persona boundaries) exist in the system and run on every tool call, but they are infrastructure, not the contribution of this work, and we treat them only briefly.

2 The Pre-Write Declaration Protocol

2.1 Mechanics

Each declaration writes the following record to `cog_step.json`, under a lock protocol (atomic replace, TTL lock, monotonic fencing token, zombie-lock cleanup) safe for concurrent windows:

⟨ phase, label, description, t_{write} , ttl = 300s, window_id, fencing ⟩

A PreToolUse hook validates the file on every Write/Edit call. The deny taxonomy:

State	Decision	Rationale
missing	deny	no stated intent
expired (>300s)	deny	stale intent is not intent
system-auto-repaired	deny	the system must never forge declarations
cross-window	allow + log	runtimes share the file by design
exempt paths (scratch, state)	allow	not product writes

2.2 Bidirectional binding

The declaration is bound to a one-line anchor in the agent’s reply (ANCHOR: + the same label). The file is machine-truth—gate-checkable; the anchor is attention-truth—it survives context compaction. Either alone drifts; together the declared intent stays visible to both the gate and the model.

2.3 Why this is an instrumentation point

The declaration stream is the freshest available representation of “what the agent believes it is doing”. Section 4 describes three subsystems that consume it. The key property: the signal is produced by the agent itself, under an enforceable discipline, at the exact moment before state changes—not reconstructed post-hoc from logs.

2.4 Compliance in practice

Anthropic’s own RFC #45427 [1] notes that deterministic tool gating in Claude Code remains an open problem. Our field observation on two runtimes: when a deny reason contains the exact command to declare, agents comply without hints. In one cross-runtime test, an agent denied with an expired declaration located the MCP declaration tool on its own (the deny text had suggested a shell command) and declared correctly: `write(denied) → cog_step_declare(phase, label) → write(allowed)`. Our interpretation: when the legitimate path is clear and cheap, agents take it; punishment-only gates train bypass behavior instead (bash redirection in place of Write/Edit is the natural bypass, and our monitors classify such commands as mutations for exactly this reason).

3 The Cognitive Loop

Six steps span one session; experience persists between sessions at each step’s storage.

	Step	When	What the system does
1	Situational awareness	session start	load last progress, standing rules, relevant knowledge
2	Task execution	before each write	declaration + gate validation (Section 2)
3	Associative learning	during dialogue	background model extracts reusable experience;
4	Abstraction	scheduled	knowledge association surfaces prior pitfalls by current op knowledge-graph consolidation; repeated patterns promoted to reusable rules
5	Context persistence	session end	summary + snapshot to disk; restored on next start
6	Trajectory update	every tool call	three-track logs with layer tags (Section 4.3)

The loop, not any single step, is the unit of design. Each step both consumes prior state and produces state for the next session—step 3’s captures feed step 4’s consolidation; step 6’s trajectory feeds the monitors; step 1’s boot load is step 5’s output from the previous session.

4 Three Consumers of the Declaration Signal

4.1 Monitor: drift, fix-loops, and escalation

Drift. Current operations are compared against task anchors drawn from state files (goal file, active context, declarations)—not from raw chat text (see Section 6 for why this matters). Low similarity prompts a single per-session notice to re-anchor.

Stuck detection. Three deterministic signals over a sliding window of the last 12 operations, all from the existing ops log, zero model calls: same file mutated ≥ 3 times (including `sed -i`, output redirection, `Set-Content`—bash-based mutation is classified as mutation, not execution); mutate→verify pairs ≥ 2 (verification = Read tool *or* running the test suite); same command failing ≥ 3 times.

Three-tier escalation.

1. **Nudge** (2 oscillations): inject a forced strategy change—stop editing; list ≥ 2 root-cause hypotheses; write a verification method for each; test the cheapest first. No blocking; once per session.
2. **Assignment** (3rd cycle): deny, with the deny reason demanding a four-part explanation: what was tried, why it failed, the root-cause hypothesis, why this attempt differs.
3. **Consultation**: the agent submits the explanation; a cheap external model (free-tier Qwen 7B) replies as a colleague—weakest assumption, likely root cause, verdict (*proceed* / *one-more-round* / *change-approach*); the opinion is injected back and a 10-minute clearance pass re-enables writes. If the reviewer is unreachable, a shorter (5-minute) pass is issued and tagged: the help channel must never become a deadlock source.

Production evidence: in the first weekend after deployment, five genuine consultations triggered during real research sessions (MATLAB probe-analysis debugging, LaTeX editing, Python plotting), with no human in the loop. In all five the agent submitted the explanation and subsequent edits followed the reviewer’s direction. We do not claim quality improvement from $n=5$; we report that the mechanism fires on real work and the agent cooperates with it.

4.2 Assistant: injection under anti-fatigue law

Four injection channels (boot load, per-turn complexity-tiered advice, post-tool targeted suggestions, knowledge association) operate under a strict filter. Every injection must pass three questions: can the agent derive this itself (then don’t say it); does it add information to this step (then don’t say it); will it still matter in ten turns (then don’t say it). Chronic-state reminders additionally carry a content-hash dedup and a 30-minute cooldown—the first implementation reminded every turn and rendered sessions unusable within minutes (Section 6).

Memory follows a three-tier discipline: standing rules always resident; memory files read on demand; background candidates pushed as suggestions. The small model only proposes candidates; the large model decides. This mirrors the division of labor that keeps suggestion quality from being capped by the proposer.

4.3 Consolidator: the data flywheel

Dual-track records. One write lands twice: a human-readable narrative and a machine-parsable block (conclusion, lesson, decision, difficulty, source). Structuring happens at write time by the working model itself—zero extra passes later, and the act of structuring reinforces the session’s own attention.

Trust grading. Every captured conclusion carries an anchor level: **human_confirmed** (the human confirmed it this conversation), **hard_gate** (verified by crossing a hard gate—compiled, tests green, ≥ 3 -round debug won), or **model_authored** (the model’s inference; lowest trust). Model-authored conclusions older than 7 days unreviewed surface in a per-turn inventory. Zombie knowledge does not silently enter the corpus.

Layer tags. Every trajectory entry carries a causality label—**reflex** (reaction to a gate denial, 15 s evidence window), **rhythm** (injection-driven, 30 s), **strategy** (stance/goal-state change, 60 s), **human** (direct instruction, 120 s)—inferred deterministically from existing logs, zero model calls; no evidence, no tag.

Why this is a flywheel. Transformers are architecturally flat; human experts reason in layers (situation, tempo, reflex). SFT on untagged trajectories reproduces the flatness—the model learns *what happened*, not *why*. Trajectories with layer tags, trust-graded conclusions, and human rulings are the prerequisite for training a model that inherits the discipline rather than the transcript. The system’s daily operation produces this corpus continuously; its purpose is not a better-monitored agent today but a model that needs less of this scaffolding tomorrow.

5 Cross-Runtime Port

The full loop (declaration gates, monitoring, assistance, consultation, consolidation) was ported to DeepSeek Harness [2]—an open-source agent runtime with a completely different extension model (in-process Cordis plugins vs. external PowerShell hooks)—as 13 plugin bundles. The port shares *state files, not code copies*: both runtimes read and write the same `cog_step.json` and consultation-pass files under the same protocol, calling the same underlying functions where possible. A declaration made through either runtime’s tooling unblocks both.

Two lessons paid for in production bugs:

1. **Data shape determines portability, not logical similarity.** The ported injection component anchored on “first 120 characters of the first turn.” On Claude Code that is the user’s task; on DeepSeek Harness the first turn contains persona text spliced in as user-role messages. Result: 76 consecutive “topic drift” alarms comparing work prompts against persona text (similarity ≈ 0.01 forever). Porting rule: dump the real message shapes before porting any handler.
2. **Internal traffic flows through the same pipeline.** The host process’s bookkeeping agent writes maintenance files through tool events; any gate must recognize and exempt non-conversational internal traffic, or the heartbeat writes eat denials daily.

Governance state is shared; behavioral telemetry is deliberately per-runtime (the monitors use a global sliding window that does not filter by session—mixing runtimes’ ops would cross-contaminate alerts).

6 Production Incidents

The system runs under the transparency it imposes on the agent; its own failures are logged with the same discipline. Selection:

Incident	Root cause	Fix / law
A deny gate never fired for one month	Output JSON malformed from day one (<code>Write-Output 'a' + \$v + 'b'</code> is argument mode in PowerShell, printing five lines); fail-open made it silent	Gate outputs must be E2E-validated with a real JSON parser, in CI—presence of output proves nothing
Reminder feature nagged every turn	Chronic conditions never clear, so an unconditioned per-turn reminder repeats forever	Content-hash dedup + 30-min cooldown + silent first run
76 consecutive false drift alarms	Anchor taken from raw first-turn text containing persona content (Section 5)	Anchors come from state files only; port against real data shapes
Knowledge base grew unbounded	Append-only, no expiry review	Trust grading; unreviewed items surface; supersede-by-similarity marks outdated

We consider the incident log part of the contribution: every entry above is also evidence that some monitoring component was watching when the system itself failed.

6.1 Retired component: symbolic-dynamics monitoring

An earlier version of this system (documented in the v2 paper and archive) monitored agent behavior through a symbolic-dynamics formulation: tool calls mapped to per-domain symbol alphabets, Shannon entropy and a transition-graph complexity index computed over rolling windows, and forbidden symbol sequences checked as subshift constraints. In production it produced no decision we acted on: the statistical signals duplicated what the simpler, deterministic stuck detectors of Section efsec:consumers already catch (same-file mutation counts, write-verify oscillation), at higher implementation cost and with baseline-calibration burden. We retired the statistical layer and kept only the regex forbidden-word gate, which is deterministic and cheap. We record this here because negative component results in self-built systems are rarely published, and because the retirement itself was only visible because the trajectory logs made the component’s activity (and inactivity) auditable.

7 Related Work

Behavioral monitoring of agents. AgentGenome [3] encodes ReAct traces into a 4-letter alphabet and mines n-grams predictive of task success; ProbGuard [4] learns DTMCs from execution traces with PAC guarantees. Both are post-hoc analyzers of behavior; our loop additionally instruments the agent’s *declared intent* in real time and closes the loop back into the session (injection, escalation, consultation). IAL-Scan [5] statically detects infinite-loop failure modes pre-deployment; our stuck detectors are the runtime counterpart. Metacognitive frameworks [6, 7] argue for separating executive control from task reasoning; our stance gears and layer tags are an engineering instantiation of that separation.

Deterministic gating. Anthropic RFC #45427 [1] documents that Claude Code PreToolUse hooks can silently fail and be bypassed, leaving deterministic tool gating open. Our declare-execute binding (declaration required, TTL-checked, non-fabricatable) is a response to exactly that gap; we describe its bypass surface honestly (bash-level writes) and mitigate behaviorally rather than gate-structurally.

Memory systems. Mem0, Zep/Graphiti, Letta and successors optimize storage and retrieval. Our consolidator is complementary and deliberately narrower: it grades *trust* and tags *causality* at capture time, because its output is training corpus, not retrieval hits. “Ask another model when stuck” exists as blog-level recipes; our contribution is making it framework-level, tiered, and trajectory-audited.

Cognitive-loop formulations. Prompt-level six-phase loops exist (e.g., six-phase-loop [8]) as system-prompt guidance with no enforcement. The difference here is enforcement: every step of our loop is backed by an external gate or daemon, and the loop’s pivot (declaration) is deny-enforced.

8 Limitations and Future Work

(1) Single-environment deployment (one researcher’s Windows workstation, two agent runtimes); no cross-site validation. (2) No quantitative evaluation: we have no labeled drift/stuck test set; the consultation count (5) is anecdotal. (3) The declaration protocol depends on hook enforcement; runtime-level bypass (direct API use without hooks) is out of scope. (4) Layer-tag inference is rule-based and coarse: it cannot distinguish “acted because of the injection” from “acted independently right after an injection”; semantic tagging awaits a labeled corpus—which the flywheel itself must first produce. (5) The flywheel’s terminal claim (that tagged corpora train better-behaved models) is a hypothesis; we will test it when open-weight models of this capability class run locally.

Data Availability

The system described here is open-source with a permanent archive DOI: <https://doi.org/10.5281/zenodo.20830497>.

Acknowledgments

We thank the open-source agent community, and the Claude Code and DeepSeek Harness teams for the runtime extension points this system is built on.

References

- [1] Anthropic community. Deterministic tool gate (RFC #45427). <https://github.com/anthropics/claude-code/issues/45427>, 2026.
- [2] DeepSeek. DeepSeek Harness: Everything is a plugin (developer preview). <https://deepseek.com/harness/>, 2026.
- [3] S. Deng. Your agent has a genome: Sequence-level behavioral analysis and runtime governance of LLM-powered autonomous agents. arXiv:2606.15579, 2026.
- [4] Y. Chen et al. ProbGuard: Proactive safety monitoring for LLM agents via probabilistic model checking. arXiv:2508.00500, 2025.
- [5] Anonymous. When agents do not stop: Static detection of infinite agent loops (IAL-Scan). arXiv:2607.01641, 2026.
- [6] K. Patel et al. Metacognition in LLMs: Foundations, progress, and future directions. arXiv:2607.11881, 2026.
- [7] Anonymous. MetaCogAgent: Metacognitive multi-agent framework with confidence-based routing and escalation. arXiv:2605.17292, 2026.
- [8] m. blakemore. six-phase-loop: A prompt-level cognitive cycle for agents. <https://github.com/mblakemore/six-phase-loop>, 2026.
- [9] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.

- [10] A. Mir. The geometry of truth: Layer-wise semantic dynamics for hallucination detection. arXiv:2510.04933, 2025.
- [11] D. Lind and B. Marcus. *An Introduction to Symbolic Dynamics and Coding*, 2nd ed. Cambridge University Press, 2021.
- [12] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv:2405.15793, 2024.